

TP n°1 Partie 2 – API REST avec PYTHON

Introduction

Cette partie du TP consiste à implémenter une **API REST en Python** permettant de gérer des données persistantes dans une **base de données relationnelle**. L'objectif est de montrer qu'après avoir compris les principes REST, leur mise en œuvre est accessible et structurée, même avec un autre langage que Java.

L'API développée gère une ressource **Student** (étudiant) et propose des opérations complètes de type **CRUD** (Create, Read, Update, Delete) avec une persistance dans **MySQL**.

1. Outils et technologies utilisés

- **Flask** : framework web léger utilisé pour créer des routes HTTP (endpoints).
 - **Flask-SQLAlchemy** : bibliothèque ORM permettant de manipuler la base de données via des objets Python plutôt que des requêtes SQL directes.
 - **MySQL** : système de gestion de base de données relationnelle.
 - **JSON** : format standard des échanges entre le client et l'API.
-

2. Connexion à la base de données

L'application est configurée pour se connecter à une base MySQL locale nommée **flaskdb**. L'accès se fait via un URI de connexion contenant :

- l'utilisateur,
- le mot de passe,
- l'hôte (localhost),
- le port (3306),

- le nom de la base.

L'ORM SQLAlchemy est ensuite initialisé afin de permettre les opérations de lecture/écriture en base de manière transparente.

3. Modélisation de la ressource “Student”

Un modèle **Student** est défini pour représenter la table correspondante dans la base de données.

Chaque étudiant possède :

- un identifiant unique (clé primaire),
- un nom,
- un âge.

Une méthode de conversion en dictionnaire est utilisée pour produire facilement des réponses au format JSON (ce qui est indispensable pour une API REST).

4. Création automatique de la table

Au démarrage de l'application, la table correspondant à la ressource Student est créée automatiquement si elle n'existe pas.

Cela permet d'éviter une création manuelle avec des commandes SQL et garantit que l'environnement de test est prêt rapidement.

5. Endpoints REST implémentés (CRUD)

5.1 Route d'accueil

Une route simple permet de vérifier que l'API fonctionne correctement. Elle renvoie un message de bienvenue.

5.2 Lecture de tous les étudiants (GET)

Un endpoint permet de récupérer la liste complète des étudiants enregistrés dans la base. La réponse est un tableau JSON contenant tous les objets Student.

5.3 Lecture d'un étudiant par identifiant (GET)

Un endpoint permet de récupérer un étudiant à partir de son ID.

Si l'ID n'existe pas, l'API retourne une réponse d'erreur avec le code HTTP approprié (ressource introuvable).

5.4 Création d'un étudiant (POST)

Un endpoint permet d'ajouter un nouvel étudiant.

Les données sont envoyées par le client au format JSON, et l'API vérifie la présence des champs obligatoires (nom et âge).

En cas de succès, l'étudiant est ajouté en base et retourné dans la réponse avec un code indiquant la création.

5.5 Mise à jour complète (PUT)

Un endpoint permet de remplacer entièrement les informations d'un étudiant existant.

Dans cette logique, tous les champs nécessaires doivent être fournis, sinon la requête est rejetée.

5.6 Mise à jour partielle (PATCH)

Un endpoint permet de modifier seulement certains champs (par exemple uniquement le nom ou uniquement l'âge).

Cette approche est plus flexible et évite de renvoyer toutes les informations si une seule valeur change.

5.7 Suppression (DELETE)

Un endpoint permet de supprimer un étudiant à partir de son identifiant.

En cas de succès, l'API confirme la suppression avec un message.

6. Gestion des réponses et des erreurs

L'API renvoie des réponses structurées :

- format JSON pour les données,
- messages d'erreur explicites,
- codes HTTP cohérents selon la situation (succès, création, erreur de données, ressource absente).

Cela respecte les bonnes pratiques REST et facilite l'utilisation de l'API côté client.